Workgroup:	Network Working Group
Internet-Draft:	draft-wullink-rpp-multi-party-authorization-00
Published:	14 November 2026
Intended Status:	Standards Track
Expires:	18 May 2027
Authors:	M. Wullink	P. Kowalik
---	---	---
	SIDN Labs	*DENIC*

Multi-Party Authorization for RESTful Provisioning Protocol (RPP)

Abstract

The traditional Registrar, Registry, and Registrant model for domain name management is evolving to include third-party service providers, such as DNS hosting providers. This document describes a generic multi-party authorization flow that allows registries to securely delegate domain management functions to accredited third-party service providers, while ensuring that the registrant's explicit consent is obtained before any RPP operations, described in [I-D.ietf-rpp-core], are executed.

TODO

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at https://datatracker.ietf.org/drafts/current/.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 18 May 2027.

Copyright Notice

Table of Contents

1. Introduction

The generic multi-party authorization flow described in this document allows registries to securely delegate RPP object operations to accredited third-party service providers, while ensuring that the registrant's explicit consent is obtained before any RPP operations, described in [I-D.ietf-rpp-core], are executed. The registry and the 3rd party MUST have a pre-established trust relationship, how this trust is established is out of scope for this document. The 3rd party does not have a direct trust relationship with the registrar, but the registrar trusts the registry to only accept requests from accredited 3rd parties.

TODO

2. Terminology

In this document the following terminology is used.

URL - A Uniform Resource Locator as defined in [RFC3986].

Resource - An object having a type, data, and possible relationship to other resources, identified by a URL.

RPP client - An HTTP user agent performing an RPP request

RPP server - An HTTP server responsible for processing requests and returning results in any supported media type.

3rd party - A service provider that is accredited by the registry to perform domain management operations on behalf of the registrant.

3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT","SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

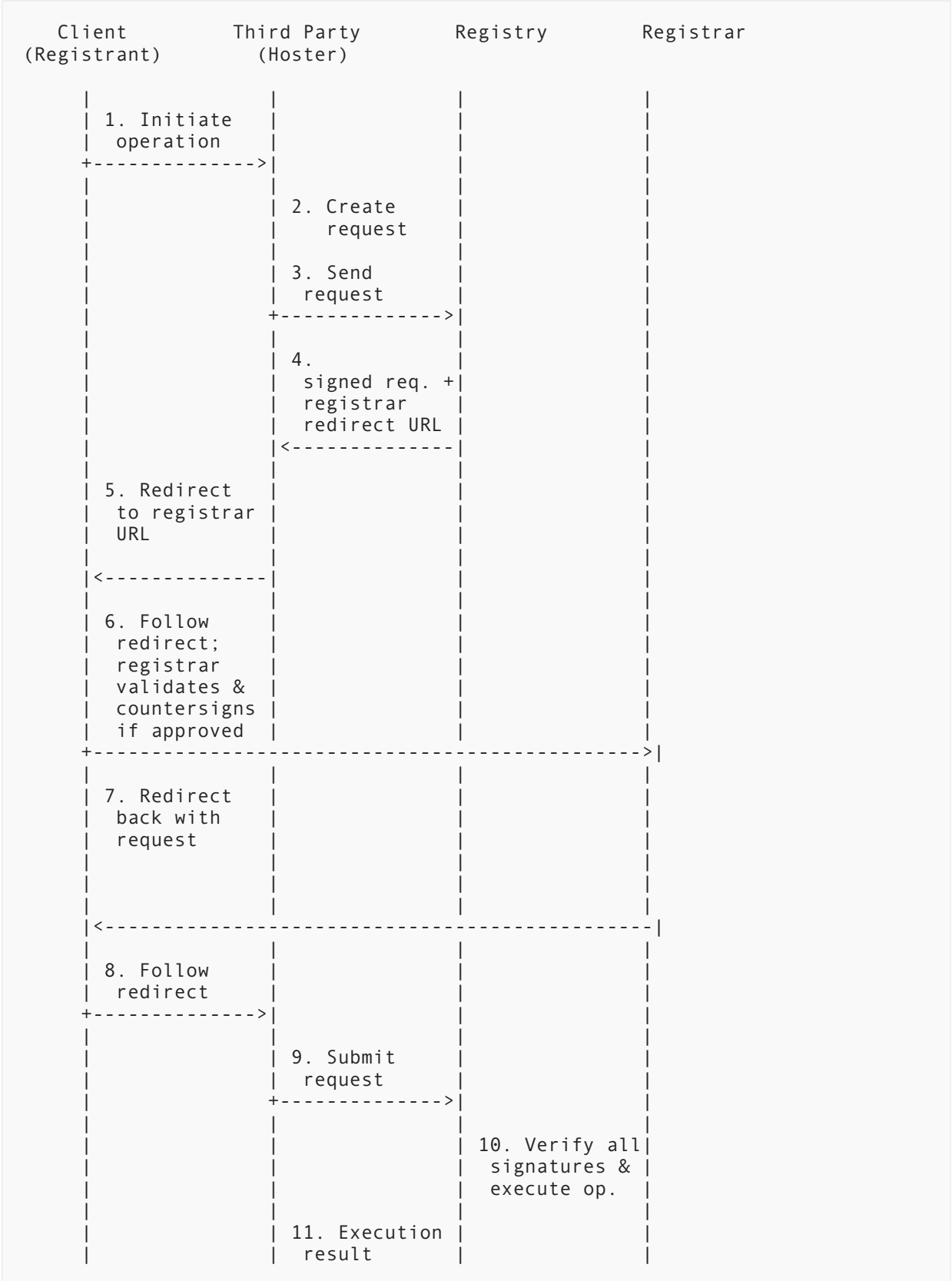
In examples, indentation and white space are provided only to illustrate element relationships and are not REQUIRED features of the protocol.

4. Architectural Overview

RPP enables registries to securely delegate domain management functions to accredited third-party service providers, starting with DNS hosting providers.

The authorization of such a delegated operation requires the explicit participation of four parties: the registrant's client, the third party performing the operation, the registry, and the registrar of record. The registry and the registrar each apply their own digital signature to the authorization request, so that the operation can only be executed once both signatures have been collected and verified. This prevents any single party from unilaterally authorizing a sensitive domain management operation.

The following diagram illustrates the multi-party authorization flow:


Client (Registrant)
Third Party (Hoster)
Registry
Registrar
1. Initiate operation
2. Create request
3. Send request
4. signed req. + registrar redirect URL
5. Redirect to registrar URL
6. Follow redirect; registrar validates & countersigns if approved
7. Redirect back with request
8. Follow redirect
9. Submit request
10. Verify all signatures & execute op.
11. Execution result

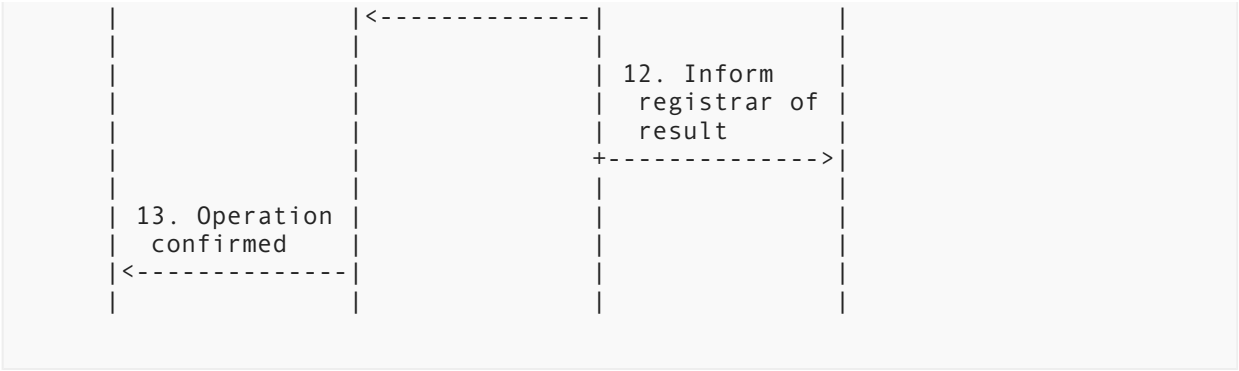


Figure 1: multi-party Authorization Flow

The steps in the diagram are as follows:

1. The client connects to the third party to initiate a domain management operation that requires authorization from the registry and the registrar.
2. The third party's backend constructs a request describing the requested operation, expressing the third party's intent to perform the operation on behalf of the registrant.
3. The third party sends the request to the RPP authorization endpoint.
4. The registry checks if the third party is accredited to act on the object and validates the request, then it signs the request. The registry returns the request, together with the URL of the registrar's web-based consent screen to which the third party must redirect the client's browser for transaction approval.
5. The third party redirects the client's browser to the registrar's consent screen at the URL provided by the registry, conveying the signed request as a parameter of the redirect.
6. The client's browser follows the redirect to the registrar. The registrar validates the signature of the registry, and presents the requested operation to the registrant for approval. If the registrant approves, the registrar adds its own signature.
7. The registrar redirects the client's browser back to the third party.
8. The client's browser follows the redirect, delivering the request signed by the registry and the registrar to the third party.
9. The third party submits the request for the RPP operation to the registry, attaching the fully-signed authorisation request in the `RPP-Multi-Party-Auth` HTTP header.
10. The registry verifies the signatures of the registry itself and the registrar. The registry executes the requested operation only if both signatures are present and valid.
11. The registry returns the result of the operation to the third party.
12. The registry informs the registrar of the result of the operation, so that the registrar can update its records accordingly.
13. The third party informs the client that the requested operation has been completed.

5. Data Objects

TODO here or in data objects doc? (now in data objects doc)

5.1. Component Data Objects

5.1.1. Public Key Data Object

TODO

5.1.2. Signature Data Object

TODO

5.1.3. Approval Data Object

TODO

5.2. Authorisation Data Object

5.2.1. Elements

This section describes the data elements of the Authorisation Data Object, as defined in [I-D.ietf-rpp-data-objects].

- `transactionType`: The type of transaction for the authorisation request. MUST be one of the values registered in the "RPP Multi-Party Transaction Types" registry Table 5. Every transaction type is associated with a specific RPP operation, and the `data` property of the authorisation request MUST contain the RPP request body for that operation.
- `id`: The identifier for the authorisation request.
- `timestamp`: The time the authorisation request was created.
- `expiration`: The time the authorisation request expires. MUST be later than `timestamp`.
- `objectId`: The identifier of the object the authorisation request pertains to.
- `requestorId`: The unique organisation identifier of the requestor.
- `requestorName`: The name of the requestor.
- `approvalUrl`: The URI to which the client should be redirected for multi-party approval. MUST only be used when the referenced organisation is a registrar or reseller supporting multi-party approval.
- `returnUrl`: The URI to which the client should be redirected after multi-party approval at the registrar. MUST only be used when the referenced organisation is a 3rd party supporting multi-party approval.
- `usage`: The usage type for the authorisation request. MUST be one of `"single-use"` or `"multi-use"`.
- `data`: The data associated with the authorisation request, containing the RPP request body for the operation identified by `transactionType`; MUST be a valid RPP Data Object, see Section 8.
- `signatures`: The digital signatures applied to the authorisation request, used to verify its authenticity and integrity, see Section 7.
- `approval`: The approval information for the authorisation request.

5.2.2. Usage of elements by parties

The Authorisation Data Object elements are set by different parties involved in the multi-party authorization flow, and read by others as needed to process or verify the transaction.

Identifier	Set by	Read by
`transactionType`	3rd party	Registry, Registrar
`id`	Registry	3rd party, Registrar
`timestamp`	Registry	3rd party, Registrar

Identifier	Set by	Read by
`expiration`	Registry	3rd party, Registrar
`objectId`	3rd party	Registry, Registrar
`requestorId`	Registry	Registrar
`requestorName`	Registry	Registrar
`approvalUrl`	Registry	3rd party
`returnUrl`	Registry	Registrar
`usage`	3rd party	Registry, Registrar
`data`	3rd party	Registry, Registrar
`signatures`	Registry, Registrar	Registry, Registrar
`approval`	Registrar	Registry, 3rd party

Table 1: Party responsible for setting and reading each data element

5.2.3. JSON Schema

This section provides normative JSON Schema definitions for the transaction types defined in this document. All schemas use JSON Schema draft 2020-12 [JSON-SCHEMA]. The schema below represents the Authorisation Data Object defined in [I-D.ietf-rpp-data-objects] and described in Section 5.2.1.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/authorisation.create",
  "$defs": {
    "authorisation.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "authorisation" },
        "transactionType": { "type": "string", "enum": ["rpp:mpa-type:domain-
ns-update", "rpp:mpa-type:domain-dnssec-update", "rpp:mpa-type:transfer"] },
        "id": { "type": "string" },
        "timestamp": { "type": "string", "format": "date-time" },
        "expiration": { "type": "string", "format": "date-time" },
        "objectId": { "type": "string" },
        "requestorId": { "type": "string" },
        "requestorName": { "type": "string" },
        "approvalUrl": { "type": "string", "format": "uri" },
        "returnUrl": { "type": "string", "format": "uri" },
        "usage": { "type": "string", "enum": ["single-use", "multi-use"] },
        "data": {
          "description": "The RPP JSON request body for the operation
identified by transactionType"
        },
        "signatures": {
          "type": "array",
          "description": "An ordered list of signatures",
```

```
          "items": {
            "type": "object",
            "properties": {
              "keyId": { "type": "string" },
              "signedAt": {
                "type": "string",
                "format": "date-time",
                "description": "The date and time at which this signature was
applied."
              },
              "signature": { "type": "string" }
            },
            "required": ["keyId", "signedAt", "signature"],
            "additionalProperties": false
          },
          "minItems": 0
        },
        "approval": {
          "type": "object",
          "properties": {
            "approved": { "type": "boolean" },
            "approvedBy": { "type": "string" },
            "reason": { "type": "string" },
            "timestamp": { "type": "string", "format": "date-time" }
          },
          "required": ["approved", "approvedBy", "timestamp"],
          "additionalProperties": false
        }
      },
      "required": ["@type", "transactionType", "id", "timestamp",
"expiration", "objectId", "requestorId"]
    }
  }
}
```

TODO currently using single schema where depending on client , registry or registrar some fields are required and some are optional. Could also define separate schemas for each party.

5.3. Organization Data Object

This specification extends the Organization type specified in [I-D.ietf-rpp-data-objects] to include two additional elements: `approvalUrl`, which is a URL that points to the registrar's web-based consent screen, and `returnUrl`, which is a URL to which the client's browser is redirected after the registrant has approved or denied the requested operation. The `approvalUrl` and `returnUrl` MUST be valid URLs, and they MUST use the HTTPS scheme.

5.3.1. Approval Redirect URL

A registrar that supports multi-party authorization MUST provide a web-based consent screen to which the 3rd party can redirect the client's browser to obtain the registrant's approval for the requested operation. The registrar MUST provide a URL for this consent screen to the registry, which is used by the registry to inform the 3rd party where to redirect the client's browser.

5.3.2. Return Redirect URL

TODO:

6. Request Processing

The request is initiated by the 3rd party, and processed by the registry and registrar. The request is signed by the registry and registrar.

6.1. Third Party

The 3rd party constructs the request, setting the `transactionType`, `objectId`, and `data` properties, and sends the request to the registry.

6.2. Registry

The registry, after validation of the request, adds the following properties to the request: `id`, `requestorId`, `requestorName`, `approvalUrl`, `returnUrl`, `timestamp`, `expiration`, `signatures`. The `id` is a unique identifier for the transaction. The `requestorId` is the identifier of the 3rd party that created the request, the `requestorName` is the public name of the requestor to be displayed in the consent screen. The `approvalUrl` is the URL where the request can be approved, the `returnUrl` is the URL to return to after approval, the `timestamp` is the time the request was created, the `expiration` is the time the request expires, the `signatures` contain the cryptographic signatures, the registry MUST add its signature of the request to the `signatures` array, it MUST be the first element in the array.

6.3. Registrar

The registrar adds the `approval` object to the request, to indicate whether the registrant and registrar have approved the transfer; at minimum, `approval.approved` and `approval.approvedBy` MUST be set. The registrar also adds its signature and public key identifier to the `signatures` array in the request, it MUST be the second element in the array. The registrar MUST verify the signature of the registry using the public key linked to the public key identifier in the request.

7. Request Signing and Verification

This section describes how each party involved in the multi-party authorization flow must cryptographically sign, and verify the authenticity and integrity of the transaction request. All parties involved in the multi-party authorization flow MUST use strong cryptography to sign and verify requests. The specific algorithms and key management practices are outside the scope of this document, but it is RECOMMENDED that parties follow best practices for cryptographic security.

Each party MUST sign only the data that it is responsible for, and MUST verify the signatures of all other parties involved in the transaction. Each party MUST include its public key identifier in the request or response, so that other parties can verify the signature.

7.1. Public Keys

A registrar that wants to participate in multi-party authorization MUST provide at least one public key to the registry. Each key is identified by a unique key identifier, which is used to verify signatures applied by the corresponding private key.

The RPP server MUST provide the current list of public keys and their identifiers to the 3rd party and the registrar, using the discovery document available at the well-known endpoint as described in [I-D.ietf-rpp-core]. The Discovery document MUST be extended to include the additional public keys and their identifiers, using the field name `registryKeys`, which is an array of JSON Web Key (JWK) objects [RFC7517], the `kid` field of each JWK object MUST be used as the key identifier.

The `use` field of each JWK object MUST be set to `sig`, indicating that the key is used for digital signatures. The `alg` field of each JWK object MUST be set to the algorithm used to sign requests, such as `RS256` for RSA signatures using SHA-256.

The following example shows a discovery document with 2 registry public keys:

```
{
  "registryKeys": [
    {
      "kty": "RSA",
      "kid": "registry-key-1",
      "use": "sig",
      "alg": "RS256",
      "n": "0vx7agoebGcQSuuPiLJXZptN9nndrQmbXEps2aiAFbWhM78LhWx4cbb....",
      "e": "AQAB"
    },
    {
      "kty": "EC",
      "kid": "registry-key-2",
      "use": "sig",
      "alg": "ES256",
      "crv": "P-256",
      "x": "f83OJ3D2xF1Bg8vub9tLe1gHMzV76e8Tus9uPHvRVEU",
      "y": "x_FEzRu9m36HLN_tue659LNpXW6pCyStikYjKIWI5a0"
    }
  ]
}
```

7.1.1. JSON Schema

TODO

7.2. Canonicalization and Signing Input

Signers MUST NOT construct the signature input by naively concatenating field values as strings. Ad hoc concatenation is ambiguous: for example, concatenating the values "ab" and "c" produces the same string as concatenating "a" and "bc", and differences in whitespace, member ordering, or numeric formatting between a signer's and a verifier's JSON serializer can cause a correctly-signed request to fail verification.

Instead, the signature input MUST be derived by applying the JSON Canonicalization Scheme (JCS) [RFC8785] to a JSON object containing exactly the fields covered by that signature, encoded as UTF-8 octets. JCS produces a single, deterministic byte string for a given set of field values, independent of the original member order or formatting, making the signature input reproducible by any conforming verifier.

A signature MUST NOT cover the signature or key identifier fields of any other party. Chaining a later signature over earlier signatures would add no additional protection. Each party MUST include its own public key identifier alongside its signature, so that other parties can verify it.

7.3. Registry

The registry MUST verify the transaction request, and copy the request to a response document while adding additional transaction related properties, and finally sign the request using its private key. The signature MUST be included in the response object.

The registry signs the JCS-canonicalized form of:

- transactionType
- id
- timestamp
- expiration
- objectId
- requestorId
- approvalUrl
- returnUrl
- data

The registry inserts the signature as an object in the `signatures` array.

7.4. Registrar

The registrar MUST verify the signature of the registry using the public key linked to the public key identifier found in the first element of the `signatures` array, and copy the request to a response document while adding additional transaction related properties, and finally sign the request using its private key. The signature MUST be included in the response object.

The registrar signs the JCS-canonicalized form of all properties signed by the registry, but it MUST include the following additional property in the signature input:

- approval

The registrar inserts the signature as an object in the `signatures` array.

Verifiers MUST independently validate each signature present in the request against this same canonical input, using the public key identified by that signature's key identifier field, and MUST reject the request if any signature does not verify.

TODO OR just have both parties sign all the fields, except the signatures array. and have the verifier check that both signatures are present and valid.

8. Transaction Types

TODO the transaction types ares defined in separate documents, this document only describes the generic flow and the signing and verification of transactions. The transaction types are defined in separate documents, which MUST be registered with IANA as described in Section 13.

This document defines three transaction types for multi-party authorization: `rpp:mpa-type:domain-ns-update`, `rpp:mpa-type:domain-dnssec-update`, and `rpp:mpa-type:transfer`. A transaction type allows access to specific RPP operations for specific objects only. Future specifications may define additional transaction types, which MUST be registered with IANA as described in Section 13.

This specification does not define a separate payload format for the RPP operation being authorized. Instead, the `data` property of a transaction request MUST contain the same JSON request body that would be sent directly to the RPP server for the corresponding operation, using the mapping rules and object representations defined in [I-D.ietf-rpp-json].

8.1. Domain Name Server Update

The `rpp:mpa-type:domain-ns-update` transaction type is used to update the DNS hosting of a domain name, by replacing the set of Host Data Objects referenced in the domain's `nameservers` property with a new set of Host Data Objects. The `objectId` property MUST contain the domain name of the target Domain Name Object.

The `data` property MUST contain a valid RPP JSON partial update request body (a JSON Patch document, as defined in the Partial Update rules of [I-D.ietf-rpp-json]), consisting of one `replace` operation for each existing Host Data Object being replaced. Each operation's `path` MUST be `/nameservers`, its `match` property MUST identify the existing Host Data Object being replaced by its `hostName`, and its `value` property MUST contain the new Host Data Object.

TODO

8.2. Domain DNSSEC Update

The `rpp:mpa-type:domain-dnssec-update` transaction type is used to update the DNSSEC DS records of a domain name, by replacing the set of DS records referenced in the domain's `dns` property with a new set of DS records. The `objectId` property MUST contain the domain name of the target Domain Name Object.

The `data` property MUST contain a valid RPP JSON partial update request body (a JSON Patch document, as defined in the Partial Update rules of [I-D.ietf-rpp-json]).

TODO

8.3. Domain Transfer

The `rpp:mpa-type:transfer` transaction type is used to transfer the sponsorship of a domain to another registrar. The `objectId` property MUST contain the domain name of the target Domain Name Object, and the `data` property MUST contain a valid Transfer Process Object create request body, as defined in [I-D.ietf-rpp-json], for the transfer operation described in [I-D.ietf-rpp-core].

TODO

9. HTTP

9.1. Endpoints

The following non normative table lists RPP endpoints related to authorisation processes, each derived by applying the rules defined in section "HTTP Mapping Rules" in [I-D.ietf-rpp-core].

Operation	HTTP Method	URL path
Authorisation: create	`"POST"`	`"/{collection}/{id}/processes/authorisationProcesses"`
Authorisation: read	`"GET"`	`"/{collection}/{id}/processes/authorisationProcesses/latest"`
Authorisation: read	`"GET"`	`"/{collection}/{id}/processes/authorisationProcesses/{id}"`
Authorisation: list	`"GET"`	`"/{collection}/{id}/processes/authorisationProcesses"`

Table 2

TODO

9.2. Request Header

The `RPP-Multi-Party-Auth` MUST be used by the 3rd party when submitting an authorised RPP request to the registry. Without this header, the registry MUST reject the request with a `400 Bad Request` response.

The value of the `RPP-Multi-Party-Auth` header is a base64 encoded string that represents the signed approval request, the header uses the following ABNF syntax:

```
Approval = "RPP-Multi-Party-Auth" ":" SP approval-token
approval-token = 1*CHAR
```

Example HTTP request using the `RPP-Multi-Party-Auth` header in a domain update operation, to update the DNS records for a domain object:

```
PATCH /domainNames/test1.example HTTP/1.1
RPP-Multi-Party-Auth: <rpp-mpa-token>
Authorization: Bearer <access-token>
Content-Type: application/json

**TODO** messagebody here
```

TODO do we want to base64 encode the approval token? It is a JSON object, so it can contain characters that are not allowed in HTTP headers. Base64 encoding would make it safe to include in the header, but it would also make it larger. We could also use a query parameter instead of a header, but that would make it visible in logs and browser history.

9.3. Return Header

The 3rd party MUST include the `RPP-Multi-Party-Return` header in the HTTP response when redirecting the registrant browser to the registrar approval page. The value contains the URL where the registrant's browser should be redirected after the request is approved by both the registrar and the registrant. The header uses the following ABNF syntax:

```
Return = "RPP-Multi-Party-Return" ":" SP return-url
return-url = 1*CHAR
```

10. Examples

10.1. Key provisioning

Each party involved in the multi-party authorization flow MUST provide at least 1 public key at the registry. Public keys are provisioned by adding a Public Key Object to the `publicKeys` property of the party's Organisation Data Object [I-D.ietf-rpp-data-objects], using an RPP JSON partial update (JSON Patch, as defined in the Partial Update rules of [I-D.ietf-rpp-json]). Since `publicKeys` is a `DictionaryComposition[Public Key Object]`, adding a new key MUST be done using an `add` operation whose `path` targets the new key identifier directly; the `match` property is not used, as it only applies to array-valued properties.

The following example shows a partial update request adding an RSA public key with identifier `registrar-key-3` to the `publicKeys` property of the Organisation Data Object with id `ORG-12345`:

```
PATCH /organisations/me HTTP/1.1
Authorization: Bearer <access-token>
Content-Type: application/json
[
  {
    "op": "add",
    "path": "/publicKeys",
    "value": {
      "registrar-key-3": {
          "@type": "publicKey",
          "type": "RSA",
          "alg": "RS256",
          "n": "0vx7agoebGcQSuuPiLJXZptN9nndrQmbXEps2aiAFbWhM78LhWx4cbb....",
          "e": "AQAB"
        }
    }
  }
]
```

TODO do we want to define "me" as a special identifier for the org of the current loggedin user, makes things easier for clients, will need to update core doc.

10.2. Redirection URL management

The registrar's `approvalUrl` and the 3rd party's `returnUrl` of their respective Organisation Data Object MUST be set, and MUST be valid URLs. Updating these URLs is done using an RPP JSON partial update of the Organisation Data Object, using a `replace` operation for each URL. The following example shows a partial update request to set the `approvalUrl` of the registrar's Organisation Data Object with id `ORG-12345`:

```
PATCH /organisations/ORG-12345 HTTP/1.1
Authorization: Bearer <access-token>
Content-Type: application/json
[
  {
    "op": "replace",
    "path": "/approvalUrl",
    "value": "https://registrar.example/consent"
  }
]
```

TODO

10.3. Domain Name Server Update Request

Example 3rd party request to registry to replace the nameservers of a domain object with a new set of Host Data Objects. The `data` property contains an RPP JSON partial update (JSON Patch) request body as defined in [I-D.ietf-rpp-json], with one `replace` operation for the existing nameservers. The example below shows a request to update the domain `test1.example`, replacing the existing nameservers with `ns1.dns.example` and `ns2.dns.example`:

```
{
  "@type": "authorisation",
  "transactionType": "rpp:mpa-type:domain-ns-update",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "data": [
    {
      "op": "replace",
      "path": "/nameservers",
      "value": [
        { "@type": "host", "hostName": "ns1.dns.example" },
        { "@type": "host", "hostName": "ns2.dns.example" }
      ]
    }
  ]
}
```

Example Registry Response: **TODO**

Example Registrar Response: **TODO**

10.4. Domain DNSSEC Update Request

Example 3rd party request to registry to replace the DNSSEC DS records of a domain object with a new set of DS records. The `data` property contains an RPP JSON partial update (JSON Patch) request body as defined in [I-D.ietf-rpp-json], with one `replace` operation for the existing DS records. The example below shows a request to update the domain `test1.example`, replacing the existing DS records with a new DS record with key tag `12345`:

```
{
  "@type": "authorisation",
  "transactionType": "rpp:mpa-type:domain-dnssec-update",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "data": [
    {
      "op": "replace",
      "path": "/dns",
      "value": {
        "@type": "dnsData",
        "records": [
            { "@type": "dnsRecord",
              "name": "@",
              "ttl": 3600,
              "type": "ds",
              "rdata": { "keyTag": 12345,
                         "algorithm": 8,
                         "digestType": 2,
                         "digest": "ABCDEF1234567890"
                       }
            }
        ]
      }
    }
  ]
}
```

Example Registry Response: **TODO**

Example Registrar Response: **TODO**

10.5. Domain Transfer Request

Example 3rd party request to registry to transfer the management of a domain object, signed by the 3rd party. The `data` property contains a Transfer Process Object create request body as defined in [I-D.ietf-rpp-json]:

```
{
  "@type": "authorisation",
  "transactionType": "rpp:mpa-type:transfer",
  "id": "TR-12345",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "data": {
    "@type": "transferProcess",
    "transferDir": "pull"
  }
}
```

Example registry response to the 3rd party to transfer the management of a domain object, signed by the registry, this response is sent to the losing registrar to request the registrant's approval for the transfer:

The registry adds the following fields to the request: `id`, `requestorId`, `requestorName`, and `signatures`. The `id` is a unique identifier for the transaction, the `requestorId` is the identifier of the 3rd party that created the request, the `requestorName` is the public name of the requestor to be displayed in the consent screen, and the `signatures` array contains the registry's signature and public key identifier, as the first element in the array.

```
{
  "@type": "authorisation",
  "transactionType": "rpp:mpa-type:transfer",
  "id": "TR-12345",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "requestorId": "ORG-3RDPARTY-1",
  "requestorName": "Example Registrar",
  "data": {
    "@type": "transferProcess",
    "transferDir": "pull"
  },
  "signatures": [
    {
      "keyId": "my-registry-key-1",
      "signedAt": "2027-06-01T12:00:00Z",
      "signature": "yyyyy"
    }
  ]
}
```

Example response from losing registrar to the 3rd party to transfer the management of a domain object, signed by the registrar; the following `approval` object is added to the request, to indicate whether the registrant and registrar have approved the transfer. The registrar also adds its signature and public key identifier to the `signatures` array in the request.

This response is sent to the registry to request execution of the transfer:

```
{
  "@type": "authorisation",
  "transactionType": "rpp:mpa-type:transfer",
  "id": "TR-12345",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "requestorId": "ORG-3RDPARTY-1",
  "requestorName": "Example Registrar",
  "approval": {
    "approved": true,
    "approvedBy": "registrar-admin@example-registrar.example",
    "timestamp": "2027-06-01T12:05:00Z"
  },
  "data": {
    "@type": "transferProcess",
    "transferDir": "pull"
  },
  "signatures": [
    {
      "keyId": "my-registry-key-1",
      "signedAt": "2027-06-01T12:00:00Z",
      "signature": "yyyyy"
    },
    {
      "keyId": "my-registrar-key-1",
      "signedAt": "2027-06-01T12:05:00Z",
      "signature": "zzzzz"
    }
  ]
}
```

After the registry verifies the signatures of the registry and registrar, and confirms that `approval.approved` is set to `true`, it executes the transfer operation and returns the result to the 3rd party.

11. Security Considerations

The registrant's explicit consent is required for any RPP operation to be executed. The registrant's consent is obtained through the registrar's web-based consent screen, which is presented to the registrant by the registrar. The registrar MUST ensure that the registrant's consent is obtained before any RPP operation is executed.

The registry MUST ensure that the registrant is the owner of the domain name for which the RPP operation is requested, and that the registrant's consent is obtained before any RPP operation is executed.

The registry MUST ensure that the registrar is the current sponsoring registrar of record for the domain name for which the RPP operation is requested, the sponsoring registrar may have changed between the time the request is created and the time the RPP operation, using the authorization request, is executed.

The registry MUST check if the authorization request used to request an RPP operation is correctly signed by the registry itself and the registrar, and that the request is valid and not expired. The registry MUST reject any request with a missing or invalid signature, or

that is invalid or expired. The Registry MUST check that the authorization request is used only once, when the `use` property is set to `single-use`. The registry MUST ensure that the authorization request is used only for the specific RPP operation for which it was created, and that the request is not used to authorize any other operation.

Signature verification MUST be performed locally by each verifying party using the public key of the signer, rather than by querying the signer's system at verification time. In particular, the registrar MUST verify the registry's signature (and the registry's verification of the 3rd party's signature) using the registry's public key, and MUST NOT rely on a live call back to the registry to confirm that a signature is valid. The registry already re-verifies all signatures, when the fully-signed request is submitted for execution (see Section 4).

This document does not specify any authorization and authentication mechanisms for the 3rd party, registry, and registrar to secure the HTTP endpoints used to exchange requests and responses. It is RECOMMENDED that parties follow best practices for securing HTTP endpoints, such as using TLS for end-to-end encryption, and OAUTH 2.0 [RFC6749] for authorization.

What cryptographic algorithms and key management practices are used to sign and verify requests is outside the scope of this document, but it is RECOMMENDED that parties follow best practices for cryptographic security.

TODO

12. RPP result codes

This specification defines new RPP result codes for multi-party authorization, used to indicate the result of creating an authorization request, and of an RPP operation that was requested using a multi-party authorization request. These result codes follow the result code classes defined in [I-D.ietf-rpp-core]: `14xxx` for client errors and `15xxx` for server errors. As described in [I-D.ietf-rpp-core].

12.1. Client Errors

The following client error result codes (class `14xxx`) are defined in this document:

RPP Result Code	HTTP Status Code	Description
14001	400 Bad Request	The `transactionType` is missing, unknown, or not registered in the "RPP Multi-Party Transaction Types" registry Table 5.
14002	400 Bad Request	The `data` property does not conform to the RPP JSON request body expected for the given `transactionType`.
14003	404 Not Found	The `objectId` does not identify an existing object.
14004	403 Forbidden	One or more required signatures are missing or fail cryptographic verification.
14005	400 Bad Request	A `keyId` referenced in the request does not match any public key currently provisioned by the identified party.

RPP Result Code	HTTP Status Code	Description
14006	403 Forbidden	The 3rd party is not accredited by the registry to perform the requested operation on the referenced object.
14007	403 Forbidden	The identified registrar is not the current sponsoring registrar of record for the referenced object.
14008	400 Bad Request	The `expiration` timestamp of the request has already passed.
14009	409 Conflict	The `id` of the request has already been used by a previously completed or in-progress transaction.
14010	403 Forbidden	The registrant did not approve the requested operation at the registrar's consent screen.

Table 3: RPP multi-party authorization client error result codes

12.2. Server Errors

The following server error result codes (class `15xxx`) are defined in this document:

RPP Result Code	HTTP Status Code	Description
15001	500 Internal Server Error	The registry was unable to sign the authorization request.
15002	500 Internal Server Error	The registry was unable to retrieve or validate a registrar's public key.

Table 4: RPP multi-party authorization server error result codes

13. IANA Considerations

IANA is requested to register the result codes defined in Table 3 and Table 4 in the "RPP Result codes" registry defined in [I-D.ietf-rpp-core].

IANA is requested to create a new registry for RPP multi-party transaction type identifiers, these identifiers are used to identify the type of RPP transaction used by 3rd parties to request authorization from the registry and registrar. The registry is to be called "RPP Multi-Party Transaction Types" and is to be maintained by the IETF RPP working group.

```
Name of the registry: RPP Multi-Party Transaction Types
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review
```

The following transaction types are defined in this document:

Identifier	Description
rpp:mpa-type:domain-ns-update	Update DNS hosting (nameservers) for a domain
rpp:mpa-type:domain-dnssec-update	Update DNSSEC DS records for a domain
rpp:mpa-type:transfer	Transfer a domain to another registrar

Table 5: RPP multi-party authorization transaction types

TODO

14. Internationalization Considerations

TODO

15. Privacy Considerations

The registrant personally identifiable information (PII) is not included in the multi-party authorization flow, except for the registrant's explicit consent obtained through the registrar's web-based consent screen.

TODO

16. Change History

TODO

17. References

17.1. Normative References

[I-D.ietf-rpp-core] Wullink, M. and P. Kowalik, "RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-core-00, 6 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-core-00>.

[I-D.ietf-rpp-data-objects] Kowalik, P. and M. Wullink, "RESTful Provisioning Protocol (RPP) Data Objects", Work in Progress, Internet-Draft, draft-ietf-rpp-data-objects-01, 3 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-data-objects-01>.

[I-D.ietf-rpp-json] Wullink, M. and P. Kowalik, "JSON for RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-json-00, 6 July 2026, <https://datatracker.ietf.org/doc/html/draft-ietf-rpp-json-00>.

[RFC2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <https://www.rfc-editor.org/info/rfc2119>.

[RFC3986] Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <https://www.rfc-editor.org/info/rfc3986>.

[RFC6749] Hardt, D., Ed., "The OAuth 2.0 Authorization Framework", RFC 6749, DOI 10.17487/RFC6749, October 2012, <https://www.rfc-editor.org/info/rfc6749>.

[RFC7517] Jones, M., "JSON Web Key (JWK)", RFC 7517, DOI 10.17487/RFC7517, May 2015, <https://www.rfc-editor.org/info/rfc7517>.

[RFC8785] Rundgren, A., Jordan, B., and S. Erdtman, "JSON Canonicalization Scheme (JCS)", RFC 8785, DOI 10.17487/RFC8785, June 2020, <https://www.rfc-editor.org/info/rfc8785>.

17.2. Informative References

[JSON-SCHEMA] JSON Schema, "JSON Schema: A vocabulary for describing JSON Data", 2020, <https://json-schema.org/draft/2020-12/json-schema-core>.

Acknowledgements

TODO

Authors' Addresses

Maarten Wullink
SIDN Labs
Email: maarten.wullink@sidn.nl
URI: https://sidn.nl/

Pawel Kowalik
DENIC
Email: pawel.kowalik@denic.de
URI: https://denic.de/